

Nebula: Reliable Low-latency Video Transmission for Mobile Cloud Gaming

Ahmad Alhilal
Tristan Braud
aalhilal@ust.hk;braudt@ust.hk
Hong Kong University of Science and
Technology
Hong Kong

Bo Han
bohan@gmu.us
George Mason University
Fairfax, Virginia, United States

Pan Hui
panhui@cse.ust.hk
Hong Kong University of Science and
Technology
Hong Kong

ABSTRACT

Mobile cloud gaming enables high-end games on constrained devices by streaming the game content from powerful servers through mobile networks. Mobile networks suffer from highly variable bandwidth, latency, and losses that affect the gaming experience. This paper introduces Nebula, an end-to-end cloud gaming framework to minimize the impact of network conditions on the user experience. Nebula relies on an end-to-end distortion model adapting the video source rate and the amount of frame-level redundancy based on the measured network conditions. As a result, it minimizes the motion-to-photon (MTP) latency while protecting the frames from losses. We fully implement Nebula and evaluate its performance against the state-of-the-art techniques and latest research in real-time mobile cloud gaming transmission on a physical testbed over emulated and real wireless networks. Nebula consistently balances MTP latency (<140 ms) and visual quality (>31 dB) even in highly variable environments. A user experiment confirms that Nebula maximizes the user experience with high perceived video quality, playability, and low user load.

CCS CONCEPTS

• **Networks** → *Cross-layer protocols*; • **Computer systems organization** → **Reliability**; **Real-time system architecture**.

KEYWORDS

Mobile Cloud Gaming, Forward Error Correction, Adaptive Rate.

ACM Reference Format:

Ahmad Alhilal, Tristan Braud, Bo Han, and Pan Hui. 2022. Nebula: Reliable Low-latency Video Transmission for Mobile Cloud Gaming. In *Proceedings of the ACM Web Conference 2022 (WWW '22)*, April 25–29, 2022, Virtual Event, Lyon, France. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3485447.3512276>

1 INTRODUCTION

Cloud gaming enables high-quality video gaming on lightweight clients, supported by powerful servers in the cloud [31]. As a highly

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WWW '22, April 25–29, 2022, Virtual Event, Lyon, France

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9096-5/22/04...\$15.00

<https://doi.org/10.1145/3485447.3512276>

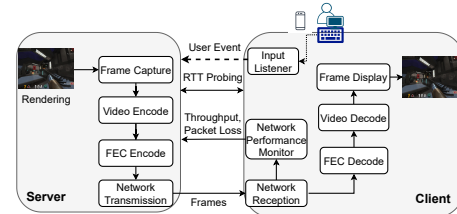


Figure 1: Nebula minimizes latency while maximizing mobile cloud gaming video quality by performing joint source rate and FEC redundancy control over unpredictable mobile wireless networks.

interactive multimedia application, cloud gaming requires reliable and low-latency network communication [7]. Mobile cloud gaming (MCG) leverages the pervasivity of mobile networks to serve video game content on constrained mobile devices. Such networks exhibit unpredictable variations of bandwidth, latency, jitter, and packet losses, significantly impacting the transmission of game content.

Currently, most commercial cloud gaming platforms target sedentary gaming at home. These platforms often prioritize low latency over video quality through protocols such as WebRTC [5, 9]. Although WebRTC minimizes transmission latency, the resulting low video quality can have a detrimental effect on the users' quality of experience (QoE)¹. Hence, improving the visual quality while preserving low latency remains a core challenge. Besides, traditional cloud gaming targets wired or controlled wireless network connections, that feature high bandwidth, low latency, and stable performance. In MCG, mobile networks' inherent unpredictability further complicates the transmission of real-time content. Sudden bandwidth drops reduce the amount of data that can be transmitted, while packet losses cause distortions in the decoded video flow. Cloud providers over public WANs present higher latency variation compared to providers with private network infrastructure [8]. Adaptive bit rate (ABR) and forward error correction (FEC) have been widely used to minimize the influence of bandwidth variations and packet loss on video transmission [25]. However, most ABR solutions suffer from inaccurate estimations of the available bandwidth [3], while FEC comes at the cost of high overhead. Besides, these two schemes have rarely been combined to optimize network usage and improve the QoE.

In this paper, we introduce Nebula, the first end-to-end framework combining *video distortion propagation modelling* with *adaptive frame-level FEC* and *joint video encoding bitrate and FEC redundancy control* for MCG over mobile networks (see Figure 1). Nebula

¹<https://www.windowcentral.com/early-tests-call-google-stadias-picture-quality-question-and-thats-shame>

relies on a mathematical model of video distortion that considers, for the first time, the propagation of errors among multiple game frames. As such, contrary to existing solutions that apply FEC to the entire group of picture (GoP) [30, 32], Nebula applies frame-level FEC and unequally protects GoP frames prioritized by their proximity to the GoP's intra-frame. Nebula thus minimizes the end-to-end latency while significantly attenuating the effect of packet loss and the propagation of distortion in the GoP. Nebula also integrates source rate control to mitigate the effect of bandwidth variation.

We evaluate Nebula against standard TCP Cubic- and WebRTC-based video streaming solutions, and recent research on video streaming (Buffer-Occupancy, BO) [18, 19] and mobile cloud gaming (ESCOT) [32]. In both emulated environments and in-the-wild, Nebula balances motion-to-photon (MTP) latency and visual quality. TCP Cubic may present a higher visual quality, but the MTP latency shoots up with variable bandwidth. WebRTC keeps the MTP latency low at the cost of significant degradation of the visual quality, especially on networks with high jitter. BO and ESCOT critically affect video quality and latency, respectively. A user study performed with 15 participants over a real-life WiFi network confirms the benefits of Nebula on the QoE. Nebula results in higher perceived video quality, playability, and user performance while minimising the task load compared to all other streaming solutions.

Our contribution is fourfold:

- We propose the *first model of end-to-end distortion* that accounts for the error propagation over a GoP.
- We introduce Nebula, an end-to-end framework for joint source/FEC rate control in MCG video streaming.
- We implement Nebula and the typical transmission protocols within a functional cloud gaming system.
- We *evaluate* Nebula over a physical testbed through extensive experiments and a user study. Nebula balances low-latency with resilience to distortion, leading to a good QoE.

2 BACKGROUND AND RELATED WORKS

Although few works target MCG specifically, there is significant literature on real-time mobile video streaming. This section summarizes the major works on both video streaming and MCG systems.

2.1 Low-latency Mobile Video Streaming

FEC leads to high decoding delay when applied to full GoPs [30, 32]. Xiao et al. [34] use randomized expanding Reed-Solomon (RS) code to recover frames using parity packets of received frames. They propose an FEC coding scheme to group GoPs as coding blocks, reducing the delay compared to GoP-based FEC [33]. However, they do not address burst packet loss. Yu et al. [35] study the effects of burst loss and long delay on three popular mobile video call applications. The study highlights the importance of conservative video rate selection and FEC redundancy schemes for better video quality. Frossard et al. [11] adapt the source rate and FEC redundancy according to the network performance. Wu et al. [29] propose flow rate allocation-based Joint source-channel coding to cope with burst and sporadic packet loss over heterogeneous wireless networks. Although these works employ FEC for video transmission, they do not address the low-latency requirements of MCG.

Congestion and overshooting are key sources of loss and distortion. WebRTC controls congestion using Google Congestion Control (GCC) algorithm. GCC adapts the sending bitrate based on the delay at the receiver and the loss at the sender [4, 20]. As a result, WebRTC favors real-time transmission to video quality. On the other hand, TCP Cubic [15] is one of the most widely deployed TCP variants in mobile networks that does not use latency as a congestion signal. TCP Cubic is sensitive to large buffers in cellular networks leading to bufferbloat [13]. The Buffer-Occupancy (BO) algorithm [18, 19] selects the video rate based on the playback buffer occupancy. BO keeps a reservoir of frames with minimum quality and requests a lower rate upon low buffer occupancy signals. Such buffer-based technique encounters either low visual quality or lengthy latency, especially in extremely variable settings [19].

Although these works address low-latency video streaming, most do not consider the high interactivity of MCG and its stringent constraints on video quality, latency, and jitter. As such, there is a need for MCG-specific content transmission frameworks that address the two-way transmission of real-time data.

2.2 Mobile Cloud Gaming

A few works target specifically MCG due to its novelty. Provision of MCG is challenging owing to the high transmission rate and the limited resources. Chen et al. [6] use collaborative rendering, progressive meshes, and 3D image warping to cope with provisioning issues and improve the visual quality. Guo et al. [14] propose a model to meet players' quality requirements by optimizing the cloud resource utilization with reasonable complexity. Outatime [21] renders speculative frames that are delivered one RTT ahead of time, offsetting up to 120ms of network latency. Wu et al. [28] distribute the game video data based on estimated path quality and end-to-end distortion. In another work, they adjust the frame selection dynamically and protect GoP frames unequally using forward error correction (FEC) coding [31]. They refine the distortion model in [32] to support both source and channel distortion. Fu et al. [12] investigate the source and channel distortion over LTE networks. All the above works apply GoP-level FEC, allowing them to cancel out error propagation at the cost of high motion to photon latency. In this work, we consider for the first time the distortion propagation across the GoP and adapt the frame-level FEC accordingly. As such, Nebula improves the quality of interaction and preserves the video quality beyond conventional (WebRTC and TCP Cubic) and research (ESCOT, BO) cloud gaming systems performance.

3 MODELLING THE JOINT SOURCE/FEC RATE PROBLEM

MCG relies on lossy video compression and protocols, leading to distortion in the decoded video. After justifying the need for joint source/FEC rate adaption, we refine existing distortion models to account for error propagation within a GoP, before formulating the joint source/FEC rate optimization problem.

3.1 Joint Source/FEC Rate Problem

MCG is a highly interactive application that promises high-quality graphics on mobile devices. As such, a key metric of user experience is the MTP latency, defined as the delay between a user action and

its consequences on the display. In mobile networks, ensuring a low MTP latency with a high visual quality is the primary challenge.

FEC minimizes the impact of packet loss. However, the redundancy introduces significant overhead that increases latency and reduces the available bandwidth. Moreover, network congestion often leads to burst of losses [32], for which FEC is often ineffective or even detrimental [36]. Although FEC eliminates the retransmissions, most recent works [30–32] on FEC for cloud gaming apply FEC on the entire GoP, which either significantly increase the motion-to-photon latency (all GoP frames must be decoded before display), demand higher bandwidth, or decreases the visual quality. Besides, any burst loss can prevent the recovery of the entire GoP.

We combine rate control with FEC redundancy to mitigate congestion and residual packet loss and adapt the encoding bitrate. To minimize the MTP latency, we perform FEC at frame level instead of GoP level and vary the amount of redundancy based on the position of the frame in the GoP.

3.2 Distortion Propagation Model

End-to-end distortion represents the difference between the original frame at the sender, and the decoded frame at the receiver, and is represented by the Mean Square Error (MSE):

$$MSE = \frac{1}{KMN} \sum_{k=1}^K \sum_{m=1}^M \sum_{n=1}^N (f_k(m, n) - \hat{f}_k(m, n))^2 \quad (1)$$

where K is the number of frames with resolution $M \times N$. $f_k(m, n)$ is the original pixel value and $\hat{f}_k(m, n)$ is the reconstructed pixel value at location (m, n) of the k^{th} frame.

We consider three distortion components:

Encoder distortion D_e : information lost during encoding. Encoder distortion depends on the encoder rate R_e and empirical parameters α and R_0 so that $D_e = \frac{\theta_1}{R_e - R_0}$ [28].

Transmission distortion D_c : caused by packet loss during network transmission. Depending on the compression scheme, a packet loss may result into partial or complete frame loss. Transmission distortion is directly related to the packet loss rate Π so that $D_c = \theta_2 \Pi$ where θ_2 is an empirical parameter [28].

Decoder distortion D_d : the additional distortion that propagates over multiple frames after a network packet loss. Existing works modelling distortion for FEC encoding applies FEC to the entire GoP so as not to consider error propagation in modelling video quality. However, in frame-level FEC encoding, losing any packet corrupts the corresponding frame, directly impacting the subsequent frames in the same group of pictures (GoP). The model should thus account for such distortion propagation.

Let β denotes I-frame rate, number of Intra-frames per second. We define $D_d = \frac{\theta_3 \Pi}{\beta}$ where θ_3 is an empirical parameter. The end-to-end distortion is the summation of the three components:

$$D = D_e + D_c + D_d = \frac{\theta_1}{R_e - R_0} + \theta_2 \Pi + \frac{\theta_3 \Pi}{\beta} \quad (2)$$

Consequently, the PSNR of the pipeline is:

$$PSNR = 20 \log_{10}(MAX_I) - 10 \log_{10} \left(\frac{\theta_1}{R_e - R_0} + \left(\theta_2 + \frac{\theta_3}{\beta} \right) \Pi \right) \quad (3)$$

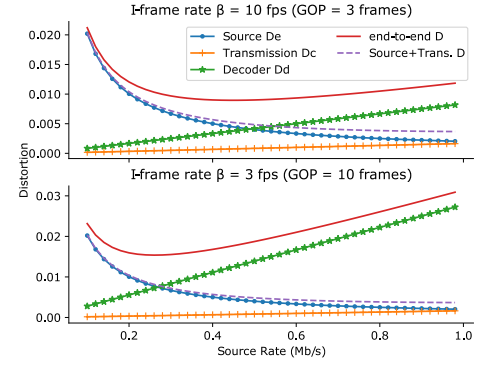


Figure 2: Source, Transmission, Decoder, and end-to-end distortion vs source video bitrate. Packet loss rate $\Pi = 0.1\%$, UDP packet size $S = 1500B$. When considering the interframe error propagation (decoder distortion), the end-to-end distortion increases with the source rate.

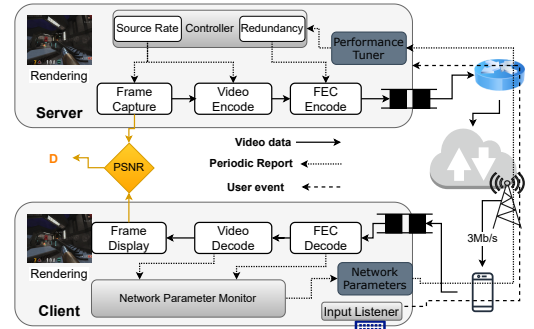


Figure 3: Detailed architecture. On the video data line, the server captures, video-encodes, FEC-encodes the game scenes, and sends them to the client. The client FEC-decodes, video-decodes the incoming stream into frames, and displays them. On the feedback line, the client measures the network performance (Client→Monitor), and sends the network parameters to the server so as to control the rate and redundancy (Server→Controller).

Figure 2 displays the source, transmission, and decoder distortion, and their combined effect on the end-to-end distortion. We also display the end-to-end distortion as represented in previous models [32]. With our model, end-to-end distortion decreases at first, until the decoder distortion takes over. The end-to-end distortion then increases linearly with the bitrate. By considering for the first time interframe error propagation, our model raises a significant source of distortion for higher source rates.

3.3 Joint source/FEC Coding Model

Distortion is a function of the available bandwidth and packet loss rate. The system objective is to minimize distortion while meeting the MCG constraints of real-time video transmission, limited and variable available bandwidth, and variable packet loss rate. We represent this objective as an optimization problem.

The available bandwidth between the MGC server and client μ determines the upper bound of the sending rate ($R_e + R_r$). The adaption scheme seeks to find an optimal rate that minimizes the

end-to-end video distortion D , given the measured network parameters at the client side (i.e., RTT , μ , Π and MTP latency), video encoding rate R_e and delay constraint T_d . In MCG, the responsiveness is critical to ensure a high QoE, with the MTP latency as the primary metric. The MTP latency is the time span between the user input and its effect on the display. It comprises user input delivery, game execution, frame rendering and capturing, video encoding, transmission, and decoding, channel encoding, and playback delay. MTP is disproportional to both sending rate $R = R_e + R_r$ and available bandwidth μ , and proportional to the queuing delay $Q_d = RTT - RTT_{min}$. The joint source and FEC coding adaptation problem for each frame can be formulated as follows.

$$\begin{aligned} \{R_e, R_r\} &= \arg \min D + \varphi MTP \\ S.t.: & D = \frac{\theta_1}{R_e - R_0} + \theta_2 \Pi + \frac{\theta_3 \Pi}{\beta} \\ & MTP = \frac{\alpha_1}{\mu} + \frac{\alpha_2}{R_e + R_r} + \alpha_3 Q_d + \alpha_4 \\ \text{Latency Cnst} & MTP \leq T_d \\ \text{Throughput Cnst} & R_e + R_r \leq \mu \end{aligned} \quad (4)$$

where φ is a hyper-parameter, T_d is the upper-bound of MTP latency (i.e., 130ms), θ_1 , R_0 , θ_2 and θ_3 are empirical parameters based on the encoder's configuration, and α_1 , α_2 , α_3 , and α_4 are parameters derived from multivariate regression with goodness of fit $R^2 = 95\%$. In the regression, we use a collective dataset of 12 experiments over both wired and wifi networks to fit the linear model. Given the number of source packets k and Π , we derive the total number of packets (sum of source and redundant packets) n as follows:

$$n = \begin{cases} \Pi(t) > 0, & \begin{cases} \max(k+1, \lceil k \cdot (1 + \omega(F-f) \cdot \Pi(t)) \rceil), & \text{cut } D_d \\ \max(k+1, \lceil (k \cdot (1 + \Pi(t))) \rceil), & \text{Otherwise} \end{cases} \\ \Pi(t) = 0, & k \end{cases} \quad (5)$$

As the I-frame rate β decreases, the decoding distortion, and thus the end-to-end distortion, increases rapidly as shown in Figure 2. The longer the error propagates, the higher the decoding distortion at the client side. To overcome the inter-frame error propagation that causes the decoding distortion, we protect the frames with a variable amount of redundancy, weighted by the frame index f within the GoP. The redundant packets $k \cdot \omega \cdot \Pi \cdot (F - f)$ ensures higher protection the closer frame f is to the I-Frame (smaller f), where $0 < \omega < 0.4$ is an empirical weight that increases or decreases with the packet loss rate. When $\omega \cdot (F - f) = 1$, all frames are protected equally, and there is no extra protection against error propagation.

3.4 Heuristic Model

Minimizing the encoding distortion D_e requires maximizing the encoding bitrate R_e . We do so heuristically by taking the maximum feasible source bitrate and the minimum redundancy rate at each

time step t as follows:

$$\begin{cases} \text{Max}(R_e) = \sum_{t=0}^T \text{Max}(R_e(t)) \\ \text{Min}(R_r) = \sum_{t=0}^T \text{Min}(R_r(t)), \forall t \in T \end{cases} \quad (6)$$

To satisfy the latency and throughput constraints in the optimization problem 4, the sending bitrate ($R = R_e + R_r$) decreases automatically once experiencing a high round trip while not exceeding the measured throughput μ at the client. At a time t , the sending rate R is upper-bounded by $\mu(t) \cdot (1 - Q_d(t))$, where $Q_d(t) = RTT(t) - RTT_{min}$ is the queuing delay, the difference between recent and min round trip time. Experimentally, satisfying the latency and throughput constraints prevents overshooting, thus satisfying the loss constraint implicitly. The formula that determines the sending bitrate R is therefore defined as follows:

$$R = \begin{cases} \max R_e(t) + \min R_r(t) < \mu(t)(1 - Q_d(t)), & 1 - Q_d(t) > 0 \\ \min R_e(t) + \min R_r(t), & 1 - Q_d(t) \leq 0 \end{cases} \quad (7)$$

With $t \in T$ is the current time, and T is the total streaming duration. t takes discrete values. In our evaluation, we choose a time interval of 1 second for a 1-minute long video game session.

The system periodically reacts to the loss rate Π , available bandwidth μ , and the MTP latency MTP by adapting R_e and R_r , avoiding overshooting and distortions that result from sporadic packet loss.

4 DISTORTION MINIMIZATION FRAMEWORK

Figure 3 illustrates our proposed end-to-end framework. This framework aims to provide MCG systems with error resiliency and achieve optimal video quality by combining source rate control and FEC coding adaptation. After providing a general description of the framework architecture, we will focus on how the adaptive FEC coding and the adaptive source coding models operate.

4.1 Framework Architecture

The framework includes two components, a controller and a parameter tuner. The **parameter tuner** receives feedback from the client at runtime, tunes parameters related to the source and FEC encoding rate, and passes them to the controller. The **controller** combines the video rate controller (to control the video encoder according to the tuned source coding rate R_e) and the redundancy rate controller (to control the RLNC coder according to the tuned redundancy rate R_r and packet size S). Each captured frame at the server-side is compressed using the VP8 codec² and split into packets encoded for network transmission using RLNC [17]. The RLNC packets are transmitted using a UDP socket to the client's mobile device. The client decodes the RLNC packets to recover the frame data, which is then decoded to recover and display the game frame.

The framework also includes the **network performance monitor** at the client-side to monitor the bandwidth, the loss rate, the network round-trip-time, and the total MTP latency. These parameters are sent periodically to the parameter tuner at the server-side.

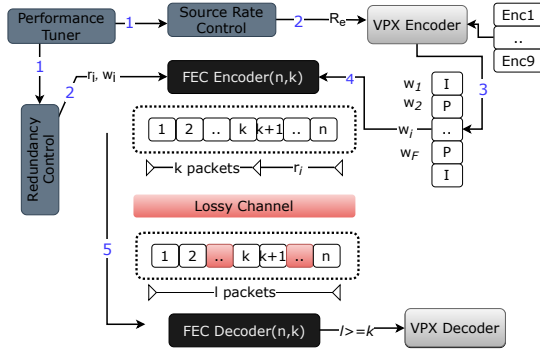


Figure 4: VPX and FEC coding. VPX encoder uses multiple encoder instances (Enc1-9) for multiple bitrates (resolutions). Performance Tuner determines encoding and redundancy rate. FEC encoder applies unequal redundancy for GoP frames (equation 5). FEC coder start decoding once $l \geq k$ packets arrived

4.2 Adaptive FEC Coder Model

Our system uses Random Linear Network Coding (RLNC) to protect the game frames against channel loss. The frame data is divided into k packets of size S and linearly combined. The RLNC encoder generates a redundancy rate R_r of $r = n - k$ redundant packets with an FEC block of size n data packets [17]. The decoder recovers the frame if any random k packets in the block are received, as shown in Figure 4. The parameter tuner extracts the packet loss information from the client's report and tunes the redundancy based on the frame index within the GoP (see Equation 5). The variable FEC coding of the GoP frames ensures lower error propagation as the frames leading to higher propagation receive higher protection.

4.3 Adaptive Source Coding Model

The bandwidth often shrinks when congestion occurs, leading to burst traffic loss [32]. Injecting more FEC-based redundant packets is ineffective to cope with congestion losses in the presence of burstiness [36]. As such, we introduce the source rate controller to control both the frequency of capturing the rendered frames and video encoding rate R_e . The parameter tuner extracts the available bandwidth and MTP latency from the client's report adjusts R_e by scaling down the game frames. It adapts to the client's bandwidth while minimizing both the distortion (see equation 4) and MTP latency and relieving the congestion. Figure 4 illustrates the operation of the source rate control and redundancy control. The VPX encoder initializes multiple instances corresponding to different bitrates, and switches the active instance based on R_e . By encoding videos several resolutions, the video encoder can dynamically adapt the bitrate of the video to the channel conditions. The video is encoded into nine resolutions: HD (1080p, 720p), High (480p, 540p), Med (360p, 376p), and Low (270p, 288p and 144p), corresponding to bitrates of (6.5 Mb/s, 4.5 Mb/s), (3 Mb/s, 2 Mb/s), (1.8 Mb/s, 1.2 Mb/s), and (1 Mb/s, 0.6 Mb/s and 0.2 Mb/s), respectively. The source rate controller selects the closest bitrate to the measured throughput μ . The resulting video dynamically changes resolution with minimal distortion as few packets are lost due to the bandwidth variations.

²<https://www.webmproject.org/code/>

5 IMPLEMENTATION

We implement the distortion minimization framework into a functional prototype system. The framework leaves multiple aspects at the discretion of the developer, that we discuss in this section.

5.1 Video and Network codecs

The framework is designed to be video and FEC codec-agnostic. However, we chose to focus on VP8 for video encoding/decoding and RLNC for FEC in our implementation.

Video codec: We use VP8 [2], an open-source codec developed by Google. Like H.264, VP8 compresses the game's frames spatially (intra-frame coding) and temporally (inter-frame coding). VP8 presents it presents similar performance as H.264 (encoding time, video quality) while being open-source. VP8 also combines high compression efficiency and low decoding complexity [1]. We rely on libvpx, the reference software implementation of VP8 developed by Google and the Alliance for Open Media (AOMedia).

FEC codec: We use Random Linear Network Coding (RLNC) ³ for channel coding. RLNC relies on simple linear algebraic operations with random generation of the linear coefficients. As such, RLNC is operable without a complex structure. RLNC can generate coded data units from any two or more coded or uncoded data units locally and on the fly [10]. The RLNC decoder can reconstruct the source data packets upon receiving at least l linearly independent packets (see Figure 4). To use RLNC encoding and decoding functionalities, we use Kodo [24], a c++ library for network/channel coding.

5.2 Client

Network Performance Monitor (NPM): This component measures the experienced packet loss rate Π , bandwidth μ , round trip time RTT , and MTP latency (MTP). The link throughput μ is computed every Δt as the net summation of the received frames' size over the net summation of frames' receiving time,
$$\mu = \frac{\sum_{f=1}^{F_s} \left(\sum_{p=2}^k S(p) \right)}{\sum_{f=1}^{F_s} t(f)},$$
 where F_s is the number of frames received secondly, f is a frame of k packets with size $S(f)$, and $t(f)$ is the elapsed time to receive it. Π is the ratio of missed packets to k . MTP is the time span between user input and playing back the corresponding frame on the client screen. The NPM smoothes these values by applying a moving average over the latest five measurements.

5.3 Server

The server measures the RTT through periodic **packet probing** [23], at regular intervals Δt .

Parameter Tuner tunes the source rate R_e and the redundancy rate R_r according to the packet loss rate Π , and network throughput μ received from the clients, and RTT . It adjusts the number of redundant packets $r = n - k$ according to the number of source packets k and Π based on equation 5. It also determines the suitable spatial resolution of the game frames and frame capturing frequency to maximize quality with a source rate R_e lower than μ .

³http://docs.steinwurf.com/nc_intro.html

Redundancy Controller reads R_r from the Parameter Tuner and updates the redundant packets for the encoded video frame. Likewise, the **Rate Controller** reads R_e to update the frame resolution.

5.4 Prototype System Implementation

We implement our system as a python program with C and C++ bindings. We adopt a multi-process approach to improve performance and prevent a delayed operation to interrupt the entire pipeline. We separate the pipeline into 3 processes (frame capture, VP8 encoding, and RLNC encoding and frame transmission) at the server and 3 processes (frame reception and RLNC decoding, VP8 decoding, display) at the client. These processes intercommunicate using managed queues. We develop a protocol on top of UDP to carry server-to-client video data and client-to-server information. Referring to figure 3, the first server process captures the frames using the mss library. The video encoding process VP8-encodes them using a custom-written wrapper to the libvpx⁴. Libvpx does not currently provide a GPU implementation, thus, encoding/decoding is performed on machines' CPU. We use the python bindings of the Kodo⁵ library to handle the RLNC operations. The server transmits packets carrying the encoded data to the client using FEC Forward Recovery Realtime Transport Protocol (FRTT) packets. The client recovers the encoded frame using the Kodo RLNC decoder. The VP8 video frames are decoded by the libvpx and displayed. The client reports the experienced network conditions periodically using the Network Performance/Parameters Report (NPR) packets. The client also listens to the user's input and sends the event information to the server. The server returns the event sequence number in the FRTT packet carrying the corresponding encoded frame, allowing the client to measure MTP latency. The server also probes the round-trip time periodically using RTTP packets (see Supplementary Materials - Figure 10).

5.5 Feasibility as a Web Service

Many implementation choices in this section are consistent with the current practice in WebRTC. Although most current web browsers provide VP8 encoding and decoding, they do not expose these functions to web applications. Instead, they provide an interface to the main functions of the libwebrtc. Similarly, we implement the core functions of Nebula as a C++ library to easily interface with web browsers with minimal logic in Python. Nebula is implemented primarily under an asynchronous fashion, exposing callbacks to applications that may use it. It can thus easily be implemented in current web browsers, and its API endpoints can be called through JavaScript with minimal modifications.

6 EVALUATION

In this section, we evaluate Nebula through both system and user experiments. We first evaluate the system in terms of objective latency (MTP) and visual quality (PSNR) measures, before inviting users to rate their experience based on subjective metrics. All experiments are performed over a controlled testbed, with both controlled and uncontrolled network conditions to ensure reproducibility.

⁴<https://github.com/webmproject/libvpx>

⁵<https://www.steinwurf.com/products/kodo.html>

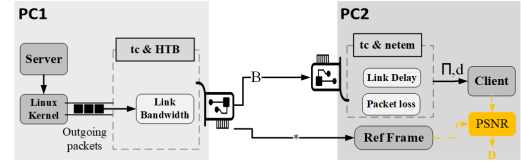


Figure 5: Experimental Testbed. It emulates a mobile network's link bandwidth, delay and packet loss using tc, netem and HTB. A reference link transmits original frame to compute the distortion D .

6.1 Evaluation Setup

We characterize Nebula over the physical testbed represented in Figure 5. This testbed is composed of PC1 as a server (Ubuntu 18.04 LTS, Intel i7-5820K CPU @ 3.30GHz) and PC2 as a client (Ubuntu 18.04 LTS, Intel(R) Core(TM) i5-4590 CPU @ 3.30GHz). Using a computer as a client simplifies the implementation of the prototype system and the system measures. The two computers are connected first through an Ethernet link to and ensure control and reproducibility. On PC1, we emulate a wireless link with bandwidth μ , latency d , and packet loss rate Π using tc⁶, NetEm⁷, and Hierarchy Token Bucket (HTB)⁸. We then connect the computers through the eduroam WiFi to perform in-the-wild experiments. On both machines, we set up the kernel HZ parameter to 1,000 to avoid burstiness in the emulated link up to 12Mb/s.

On this testbed, we compare Nebula to the following baselines (see Table 1 in C): TCP Cubic, WebRTC, Buffer Occupancy (BO) [18, 19], and ESCOT [32]. These baselines are standard in network transmission (TCP Cubic), in video streaming (WebRTC), or correspond to the latest related research works (BO, ESCOT). We implement all baselines in the pipeline described in Section 6.2 as follows. TCP Cubic is provided by the Linux Kernel. We integrate WebRTC using AioRTC⁹, that we modify to support customized source streams at the server and remote stream track of WebRTC at the client (see Supplementary Materials - Figure 12). We implement BO as a queue-based playback buffer in the pipeline. Finally, we modify Nebula to perform FEC at GoP level to implement ESCOT. We implement a user-event listener to send user's input to the server. The server executes the game logic and renders the frames upon event arrival. The architecture of the WebRTC implementation requires us to measure the visual quality in real-time. As such, we use the PSNR as it is the least computationally-intensive measure. We monitor the network conditions and all performance metrics under the same method for all experiments to ensure results consistency.

6.2 Pipeline Characterization

We first evaluate the prototype system's pipeline presented in Section 5.4. We set up the testbed with a fixed bandwidth of 20 Mb/s, latency of 50 ms, and PLR of 1%. These network conditions correspond to a typical client-to-cloud link [37], and the minimal requirements of Google Stadia. Over this testbed, we generate, encode, transmit, decode, and display a 1920×1080 video, encoded at a bitrate of 6.5 Mb/s and a frame rate of 30 FPS, with GoPs of size 10.

⁶<https://man7.org/linux/man-pages/man8/tc.8.html>

⁷<https://man7.org/linux/man-pages/man8/tc-netem.8.html>

⁸<https://linux.die.net/man/8/tc-htb>

⁹<https://github.com/aiortc/aiortc>

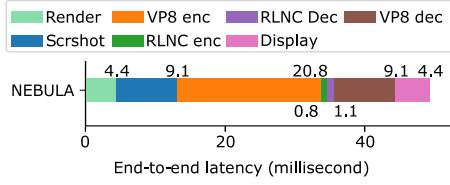


Figure 6: Latency Decomposition of the MCG Pipeline (without the additional network latency).

Figure 6 presents the latency decomposition of the pipeline without the network latency. Introducing RLNC adds a marginal delay before (0.8 ms), and after (1.1 ms) the transmission. Video encoding and decoding take the most time, 20.8 and 9.1 ms respectively as the libvpx operates solely on CPU. By leveraging hardware-level video encoding and decoding as it would be the case on mobile devices, we expect to reduce these values to 5-10 ms¹⁰, and achieve a pipeline latency below 30 ms. Nebula thus does not introduce significant latency compared to typical cloud gaming systems, while bringing loss recovery capabilities in a lossy transmission environment.

6.3 Emulated Network

To showcase the loss recovery and rate adaptation properties of the considered solutions, we emulate a wireless network with varying bandwidth over our physical testbed. On this link, we set up a bandwidth ranging between 2 Mb/s and 10 Mb/s, changing every 5 s. Such values allow us to stress the transmission solutions over variable network conditions. To ensure the reproducibility of the experiments, we generate a sequence of bandwidth allocation over time represented by the gray filled shape in Figure 8 and average the results over five runs for each solution. The link round-trip delay and loss probability remain constant over time, standing at 20 ms and 1% with probability of successive losses of 25%, respectively. Over this link, we transmit 60 seconds of a gameplay video, recorded at a resolution of 1080p, and encoded in VP8 in various resolutions by the transmission scheme.

Figure 7.a represents the MTP latency, network RTT, and average PSNR for all solutions. Only BO satisfies the requirement of MTP latency below 130 ms in such constrained environment, while Nebula and WebRTC remain close (138.6 ms and 144.8 ms respectively). TCP Cubic collapses under the abrupt bandwidth changes and presents the highest motion-to-photon latency (807.6 ms), with a high variance caused by the loss retransmission. Many frames do not reach on-time (330 ms with our pipeline) and are thus dropped by the pipeline. ESCOT also shows high MTP latency (489.8 ms) due to its GoP-level FEC encoding. As a result, ESCOT requires to receive the entire GoP before decoding, resulting in 330 ms added latency at 30 FPS with a GoP of 10. In terms of PSNR, BO can maintain such a low latency by significantly decreasing the video quality. TCP Cubic presents the highest PSNR as it does not integrate a mechanism to vary the video encoding quality. Only Nebula and WebRTC balance latency and video quality, with Nebula presenting slightly higher PSNR and lower MTP latency.

To better understand this phenomenon, we record the received rate at the client's physical interface and present the throughput of each solution in Figure 8. BO strives to eliminate video playback stalls by keeping a reservoir of frames with minimum rate. It thus

dramatically underestimates the available bandwidth, between 191 and 542 Kb/s. WebRTC consistently sends data at a rate much lower than the available bandwidth, while TCP Cubic tends to overshoot due to aggressive congestion window increase. Although Nebula and ESCOT rely on the same bandwidth estimation mechanism, ESCOT unsurprisingly tends to slightly overshoot by transmitting the entire GoP at once. By optimizing bandwidth usage, Nebula maximizes the video quality while minimising latency and losses.

6.4 Wireless Network

Following the emulated links, we proceed to an experiment on a real-life uncontrolled WiFi network. We perform the experiment on eduroam, a network commonly used by all students of the university. To ensure the statistical significance of the results, we transmit the video five times per solution and average the results. We first characterize the network by first sending a continuous flow using iPerf over 10 minutes. The network features an average bandwidth of 8.1 Mb/s (std=5.1 Mb/s) and latency of 22.7 ms (std=38.8 ms). The distribution of these values is fairly a portion of 4G LTE network traces [26]. The variability of the available bandwidth is lower than the emulated network while the latency jitter skyrockets, with up to 70 ms difference between measurement points.

Figure 7.7b represents the MTP latency, network RTT, and average PSNR for all solutions. With the more stable bandwidth, all solutions except ESCOT achieve a MTP latency below 130 ms. Similar to the previous experiment, BO achieves minimal MTP latency, with a PSNR half of the other solutions. Owing to GCC's sensitivity to delay, WebRTC collapses under the high variability of the latency, resulting in a low PSNR. ESCOT presents high MTP latency by transmitting frames by GoP-sized batches. Finally, Nebula and TCP Cubic perform similarly, with low MTP latency and high PSNR.

From the emulated network and uncontrolled WiFi experiments, only Nebula performs on average the best, balancing latency and visual quality consistently. Although TCP's behavior with varying bandwidth and the moderate loss rate is known and expected, we were surprised to notice the collapse of the visual quality of WebRTC in environments with high latency jitter, as is the case in most public wireless and mobile networks. Overall, ESCOT performs poorly due to the GoP size used in the experiments. However, a lower GoP would lead to larger videos for similar quality, and we expect the PSNR to drop with the MTP latency. Finally, BO often underestimates the bandwidth and consistently chooses the lowest quality video flow, leading to a dramatically low PSNR.

6.5 User Study

Participants and Apparatus: We perform a user study with 15 participants, aged 20 to 40. The participants are recruited on campus and are primarily students and staff. Most participants play video games at least a few times a month and have some familiarity with First Person Shooter games. The participants play OpenArena, a first-person shooting game, in a cloud gaming setting using the prototype system described in Section 5 and the transmission techniques defined in Section 6.1 over the eduroam WiFi network.

Protocol: Each participant starts with 5 minutes of free play of the game openarena executed on-device. This phase allows the participants to get familiar with the game and establish a reference of playability and visual quality. During this phase, an operator guides

¹⁰<https://developer.nvidia.com/nvidia-video-codec-sdk>

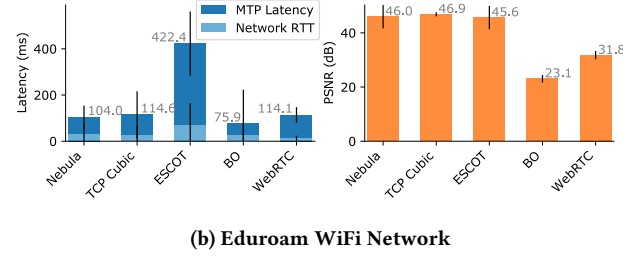
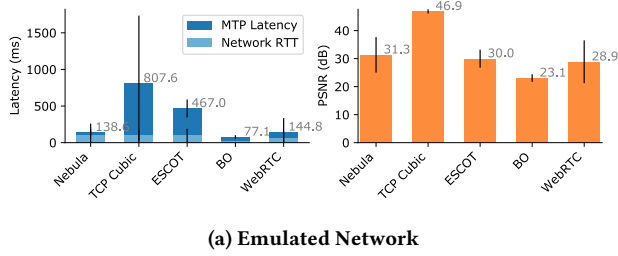


Figure 7: MTP latency, network RTT, and PSNR on emulated network (a) and eduroam WiFi network (b). On the emulated network (a), WebRTC and Nebula balance low MTP latency and high visual quality. BO achieves lower MTP latency with considerable visual quality degradation, while TCP Cubic and ESCOT suffer from massive MTP latency. On the eduroam WiFi (b), all solutions achieve a MTP latency lower than 130 ms except ESCOT. WebRTC’s visual quality collapses due to the high jitter, while TCP Cubic remains stable thanks to the higher bandwidth. In both scenarios, Nebula presents the second best visual quality and the second lowest MTP latency.

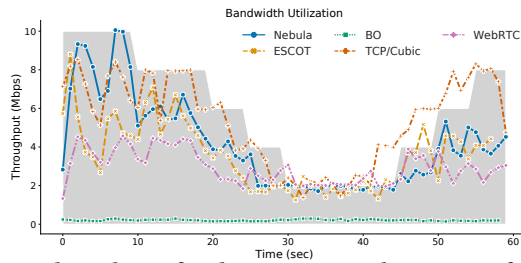


Figure 8: Throughput of each connection with variation of the link bandwidth (grey). Nebula uses the bandwidth the most efficiently. ESCOT and TCP Cubic tend to overshoot while WebRTC significantly undershoots. BO barely utilizes the available bandwidth.

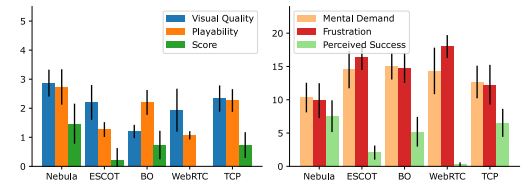


Figure 9: Users’ perception of the gaming experience under traditional cloud gaming measures (left) and task load (right). Nebula presents the highest visual quality and playability and the lowest mental demand and frustration of all solutions. Score and perceived success are also the highest with Nebula. WebRTC collapses under the high jitter of the WiFi network, resulting in low playability.

the participants and answers their questions. The participants then play the game for two minutes for each streaming method. The order in which the participants experience each streaming solution follows the balanced latin square design [27] to avoid learning and order effect. After each run, the participants fill a short questionnaire on the perceived visual quality and playability on a 5-point Likert scale (1 - bad, 2 - poor, 3 - fair, 4 - good, 5 - excellent), and a simplified version of the NASA TLX [16] survey considering the perceived mental demand, frustration, and success on a [0-20] scale. We do not disclose the streaming methods used in order not to affect the participants’ ratings.

Results: Figure 9 presents the results of the user study, with the 95% confidence intervals as error bars. Nebula results in the highest visual quality and playability, as well as the lowest mental demand and frustration. Nebula also leads to the highest objective (score) and subjective (success) performance indicators. TCP also performs

well. However, the inability to reduce the video quality in case of congestion causes transient episodes of high latency that leads the server to drop the frames, resulting in lower visual quality and playability. Similar to Section 6.4, BO leads to low latency, explaining the high playability score. However, the video quality drops so low that players have trouble distinguishing what happens on screen, resulting in high mental demand and frustration. WebRTC collapses due to the high jitter, leading to an almost-unplayable experience. Finally, the high latency of GoP-level encoding proves to be highly incompatible with a fast-paced game such as openarena. Overall, Nebula maintains latency consistently low while adapting the visual quality to the best achievable quality given the network conditions, leading to the highest user satisfaction.

This user evaluation displays results that are highly related to the type of game. Openarena is a fast-paced first-person shooter, a genre where motion-to-photon latency noticeably affects performance [22]. As such, they represent the lower bound of latency tolerance within users. The study does not consider the difference among game genres which would be an interesting future direction.

7 CONCLUSION

This paper introduced Nebula, an end-to-end framework combining per-frame adaptive FEC with source rate control to provide MCG with low-latency yet error-resilient video streaming. By combining adaptive FEC and source rate adaptation, Nebula minimizes the distortion propagation while maximizing the bandwidth usage, leading to a high end-to-end PSNR without sacrificing latency. Our system evaluation shows that Nebula balances low latency and high PSNR under all scenarios, even in highly variable conditions where other solutions (TCP Cubic and WebRTC) collapse. Under high jitter, it significantly outperforms WebRTC, the current industry standard for video transmission in cloud gaming. Our user study over a real-life public WiFi network confirms these findings, with Nebula presenting higher visual quality, playability, and user performance while requiring a lower workload than the other solutions.

We aim to focus on multiplayer streaming in our future work, where several devices share a given scene. By generating the video in multiple resolutions, our system allows distributing content to clients connected through variable network conditions. We will also continue the study of Nebula over mobile networks to evaluate the effect of mobility on the frame transmission of video games and the resulting user experience.

REFERENCES

- [1] Jim Bankoski, Paul Wilkins, and Yaowu Xu. 2011. Technical overview of VP8, an open source video codec for the web. In *2011 IEEE International Conference on Multimedia and Expo. IEEE*, Klagfurt, Austria, 1–6. <https://doi.org/10.1109/ICME.2011.6012227>
- [2] James Bankoski, Paul Wilkins, and Yaowu Xu. 2011. VP8 data format and decoding guide. *RFC* 6386 (2011), 1–304.
- [3] A. Bentalab, B. Taani, A. C. Begen, C. Timmerer, and R. Zimmermann. 2019. A Survey on Bitrate Adaptation Schemes for Streaming Media Over HTTP. *IEEE Communications Surveys Tutorials* 21, 1 (2019), 562–585. <https://doi.org/10.1109/COMST.2018.2862938>
- [4] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. 2016. Analysis and Design of the Google Congestion Control for Web Real-Time Communication (WebRTC). In *Proceedings of the 7th International Conference on Multimedia Systems (Klagenfurt, Austria) (MMSys '16)*. Association for Computing Machinery, New York, NY, USA, Article 13, 12 pages. <https://doi.org/10.1145/2910017.2910605>
- [5] Marc Carrascosa and Boris Bellalta. 2020. Cloud-gaming: Analysis of Google Stadia traffic. *CoRR* abs/2009.09786 (2020), 1–15. arXiv:2009.09786 <https://arxiv.org/abs/2009.09786>
- [6] De-Yu Chen and Magda El-Zarki. 2019. A Framework for Adaptive Residual Streaming for Single-Player Cloud Gaming. *ACM Trans. Multimedia Comput. Commun. Appl.* 15, 2s, Article 66 (July 2019), 23 pages. <https://doi.org/10.1145/3336498>
- [7] Kuan-Ta Chen, Yu-Chun Chang, Po-Han Tseng, Chun-Ying Huang, and Chin-Laung Lei. 2011. Measuring the Latency of Cloud Gaming Systems. In *Proceedings of the 19th ACM International Conference on Multimedia (Scottsdale, Arizona, USA) (MM '11)*. Association for Computing Machinery, New York, NY, USA, 1269–1272. <https://doi.org/10.1145/202298.2071991>
- [8] Lorenzo Corneo, Maximilian Eder, Nitinder Mohan, Aleksandr Zavodovski, Suzan Bayhan, Walter Wong, Per Gunningsberg, Jussi Kangasharju, and Jörg Ott. 2021. Surrounded by the Clouds: A Comprehensive Cloud Reachability Study. In *Proceedings of the Web Conference 2021 (Ljubljana, Slovenia) (WWW '21)*. Association for Computing Machinery, New York, NY, USA, 295–304. <https://doi.org/10.1145/3442381.3449854>
- [9] Andrea Di Domenico, Gianluca Perna, Martino Trevisan, Luca Vassio, and Danilo Giordano. 2021. A Network Analysis on Cloud Gaming: Stadia, GeForce Now and PSNow. *Network* 1, 3 (2021), 247–260. <https://doi.org/10.3390/network1030015>
- [10] K. Fouli, M. Médard, and K. Shroff. 2018. Coding the Network: Next Generation Coding for Flexible Network Operation. <https://www.comsoc.org/publications/ctn/coding-network-next-generation-coding-flexible-network-operation>
- [11] P. Frossard and O. Verscheure. 2001. Joint source/FEC rate selection for quality-optimal MPEG-2 video delivery. *IEEE Transactions on Image Processing* 10, 12 (2001), 1815–1825. <https://doi.org/10.1109/83.974566>
- [12] H. Fu, H. Yuan, M. Li, Z. Sun, and F. Li. 2016. Models and analysis of video streaming end-to-end distortion over LTE network. In *2016 IEEE 11th Conference on Industrial Electronics and Applications (ICIEA)*. IEEE, Hefei, China, 516–521. <https://doi.org/10.1109/ICIEA.2016.7603638>
- [13] Jim Gettys. 2011. Bufferbloat: Dark buffers in the internet. *IEEE Internet Computing* 15, 3 (2011), 96–96.
- [14] Dongyu Guo, Yiwen Han, Wei Cai, Xiaofei Wang, and Victor C. M. Leung. 2019. QoE-Oriented Resource Optimization for Mobile Cloud Gaming: A Potential Game Approach. In *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*. IEEE, Shanghai, China, 1–6. <https://doi.org/10.1109/ICC.2019.8761510>
- [15] Sangtae Ha, Injong Rhee, and Lisong Xu. 2008. CUBIC: A New TCP-Friendly High-Speed TCP Variant. *SIGOPS Oper. Syst. Rev.* 42, 5 (July 2008), 64–74. <https://doi.org/10.1145/1400097.1400105>
- [16] Sandra G Hart. 2006. NASA-task load index (NASA-TLX); 20 years later. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 50. Sage publications Sage CA: Los Angeles, CA, 904–908.
- [17] J Heide, S Shi, K Fouli, M Medard, and V Chook. 2018. Random linear network coding (rlnc)-based symbol representation. *Working Draft, IETF Secretariat, Internet-Draft draft-heide-nwrg-rlnc-02*, July 2019 (2018).
- [18] Te-Yuan Huang, Ramesh Johari, and Nick McKeown. 2013. Downton Abbey without the Hiccups: Buffer-Based Rate Adaptation for HTTP Video Streaming. In *Proceedings of the 2013 ACM SIGCOMM Workshop on Future Human-Centric Multimedia Networking (Hong Kong, China) (FhMN '13)*. Association for Computing Machinery, New York, NY, USA, 9–14. <https://doi.org/10.1145/2491172.2491179>
- [19] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. 2014. A Buffer-Based Approach to Rate Adaptation: Evidence from a Large Video Streaming Service. In *Proceedings of the 2014 ACM Conference on SIGCOMM (Chicago, Illinois, USA) (SIGCOMM '14)*. Association for Computing Machinery, New York, NY, USA, 187–198. <https://doi.org/10.1145/2619239.2626296>
- [20] Bart Jansen, Timothy Goodwin, Varun Gupta, Fernando Kuipers, and Gil Zussman. 2018. Performance evaluation of WebRTC-based video conferencing. *ACM SIGMETRICS Performance Evaluation Review* 45, 3 (2018), 56–68.
- [21] Kyungmin Lee, David Chu, Eduardo Cuervo, Johannes Kopf, Yury Degtyarev, Sergey Grizan, Alec Wolman, and Jason Flinn. 2015. Outatime: Using Speculation to Enable Low-Latency Continuous Interaction for Mobile Cloud Gaming. In *Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services (Florence, Italy) (MobiSys '15)*. Association for Computing Machinery, New York, NY, USA, 151–165. <https://doi.org/10.1145/2742647.2742656>
- [22] Shengmei Liu, Mark Claypool, Atsuo Kuwahara, Jamie Sherman, and James J Scovell. 2021. Lower is Better? The Effects of Local Latencies on Competitive First-Person Shooter Game Players. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (Yokohama, Japan) (CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 326, 12 pages. <https://doi.org/10.1145/3411764.3445245>
- [23] Matthew J. Luckie, Anthony J. McGregor, and Hans-Werner Braun. 2001. Towards Improving Packet Probing Techniques. In *Proceedings of the 1st ACM SIGCOMM Workshop on Internet Measurement (San Francisco, California, USA) (IMW '01)*. Association for Computing Machinery, New York, NY, USA, 145–150. <https://doi.org/10.1145/505202.505221>
- [24] Morten V. Pedersen, Janus Heide, and Frank H. P. Fitzek. 2011. Kodo: An Open and Research Oriented Network Coding Library. In *NETWORKING 2011 Workshops*, Vicente Casares-Giner, Pietro Manzoni, and Ana Pont (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 145–152.
- [25] Pablo Pérez, Jesús Macías, Jaime J Ruiz, and Narciso García. 2011. Effect of packet loss in video quality of experience. *Bell Labs Technical Journal* 16, 1 (2011), 91–104.
- [26] Darijo Raca, Jason J. Quinlan, Ahmed H. Zahran, and Cormac J. Sreenan. 2018. Beyond Throughput: A 4G LTE Dataset with Channel and Context Metrics. In *Proceedings of the 9th ACM Multimedia Systems Conference (Amsterdam, Netherlands) (MMSys '18)*. Association for Computing Machinery, New York, NY, USA, 460–465. <https://doi.org/10.1145/3204949.3208123>
- [27] Evan James Williams. 1949. Experimental designs balanced for the estimation of residual effects of treatments. *Australian Journal of Chemistry* 2, 2 (1949), 149–168.
- [28] J. Wu, B. Cheng, C. Yuen, Y. Shang, and J. Chen. 2015. Distortion-Aware Concurrent Multipath Transfer for Mobile Video Streaming in Heterogeneous Wireless Networks. *IEEE Transactions on Mobile Computing* 14, 4 (2015), 688–701. <https://doi.org/10.1109/TMC.2014.2334592>
- [29] Jiyuan Wu, Yanlei Shang, Jun Huang, Xue Zhang, Bo Cheng, and Junliang Chen. 2013. Joint source-channel coding and optimization for mobile video streaming in heterogeneous wireless networks. *EURASIP Journal on Wireless Communications and Networking* 2013, 1 (2013), 283.
- [30] J. Wu, C. Yuen, and J. Chen. 2015. Leveraging the Delay-Friendliness of TCP With FEC Coding in Real-Time Video Communication. *IEEE Transactions on Communications* 63, 10 (2015), 3584–3599. <https://doi.org/10.1109/TCOMM.2015.2469296>
- [31] Jiyuan Wu, Chau Yuen, Ngai-Man Cheung, Junliang Chen, and Chang Wen Chen. 2015. Enabling adaptive high-frame-rate video streaming in mobile cloud gaming applications. *IEEE Transactions on Circuits and Systems for Video Technology* 25, 12 (2015), 1988–2001.
- [32] Jiyuan Wu, Chau Yuen, Ngai-Man Cheung, Junliang Chen, and Chang Wen Chen. 2017. Streaming Mobile Cloud Gaming Video Over TCP With Adaptive Source-FEC Coding. *IEEE Trans. Circuits Syst. Video Techn.* 27, 1 (2017), 32–48.
- [33] J. Xiao, T. Tillo, C. Lin, and Y. Zhao. 2012. Dynamic Sub-GOP Forward Error Correction Code for Real-Time Video Applications. *IEEE Transactions on Multimedia* 14, 4 (2012), 1298–1308. <https://doi.org/10.1109/TMM.2012.2194274>
- [34] Jimin Xiao, Tammam Tillo, and Yao Zhao. 2013. Real-time video streaming using randomized expanding Reed–Solomon code. *IEEE transactions on circuits and systems for video technology* 23, 11 (2013), 1825–1836.
- [35] C. Yu, Y. Xu, B. Liu, and Y. Liu. 2014. “Can you SEE me now?” A measurement study of mobile video calls. In *IEEE INFOCOM 2014 - IEEE Conference on Computer Communications*. IEEE, Toronto, ON, Canada, 1456–1464. <https://doi.org/10.1109/INFOCOM.2014.6848080>
- [36] Xunqi Yu, James W Modestino, Ragip Kurceren, and Yee Sin Chan. 2008. A model-based approach to evaluation of the efficacy of FEC coding in combating network packet losses. *IEEE/ACM Transactions On Networking* 16, 3 (2008), 628–641.
- [37] Pengyuan Zhou, Wenxiao Zhang, Tristan Braud, Pan Hui, and Jussi Kangasharju. 2018. ARVE: Augmented Reality Applications in Vehicle to Edge Networks. In *Proceedings of the 2018 Workshop on Mobile Edge Communications (Budapest, Hungary) (MECOMM'18)*. Association for Computing Machinery, New York, NY, USA, 25–30. <https://doi.org/10.1145/3229556.3229564>

A SYSTEM PROTOCOL PACKETS

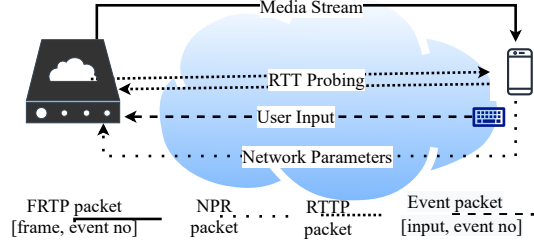


Figure 10: Illustration of media delivery and RTT probing on the forward channel and feedback reporting and user input on the reverse channel.

In this section, we illustrate Nebula's transmission protocol packets as an integral component in the **prototype system implementation** (see Section 5.4). Figure 10 illustrates the packet type used to deliver the game frames (i.e., FRTTP packet), the one used to collect reports about the network conditions (i.e., NPR), the packet type used to probe the presence of congestion (i.e., RTTP packet), and the one used to send the user input to the server (i.e., Event packet). The server sends packets carrying the encoded game frame's data to the client using FEC Forward Recovery Realtime Transport Protocol (FRTTP) packets. FRTTP packets contains the event's sequence number if the frame is rendered in response to a user's event. The client reports the experienced network conditions and MTP periodically using the Network Performance/Parameters Report (NPR) packets. It also sends user input as event identifier and number using Event packets. Moreover, the server probes the round trip time periodically using RTT Probing (RTTP) packets. Eventually, the client can compute MTP latency by subtracting the timestamp of sent event from the timestamp of a received frame having a corresponding event sequence number.

B HEURISTIC ALGORITHM DETAIL

Nebula's rate and FEC adaptation is based on discrete optimization which follows the heuristic model (see section 3.4). Algorithm 1 presents the pseudo-code of the heuristic approach. It takes as input: the ascending ordered list $list_{R_e}$, the previous encoding rate $last_{R_e}$, the size of GoP frame S_{fGoP} , and the recent MTP as well as constants, namely GoP length F and end-to-end delay upper-bound T_d . It returns the redundancy rate R_r and the video encoding rate R_e . After determining the number of packets k and computing the number redundant packets r , it computed the redundancy rate R_r . Afterwards, it determines the video encoding rate R_e heuristically (lines 8-10) based on network throughput μ and queuing delay Q_d and according to equation 7. Finally, it refines the video encoding rate based on the experienced motion-to-photon latency MTP which ensures not to exceed the latency constraint (i.e., $T_d = 130$ ms). $level$ in lines 16 or 19 determines the resultant quality level (resolution) and thus the suitable video encoding rate from the list $list_{R_e}$.

Algorithm 1 Pseudo-code of heuristic algorithm

```

1: Input:  $\mu, \beta, Q_d, \Pi, S, last_{R_e}, list_{R_e}, S_{fGoP}, MTP, F, T_d$ 
2: Output:  $R_r, R_e$ 
3:  $k \leftarrow \frac{S_{fGoP}}{S}$ 
4: Compute  $n$  using equation 5
5:  $r \leftarrow n - k$ 
6:  $R_r \leftarrow r.S.F.\beta \times \frac{8}{1024 \times 1024}$ 
7: if  $1 - Q_d > 0$  then
8:   for Each  $level_{R_e}$  do
9:     if  $list_{R_e}[level_{R_e}] + R_r \geq \mu(1 - Q_d)$  then
10:        $level \leftarrow level_{R_e} - 1$ 
11:     break
12:   end if
13: end for
14: end if
15: if  $list_{R_e}[level] \geq last_{R_e}$  &  $MTP > T_d$  then
16:    $level \leftarrow level - 1$ 
17: end if
18: if  $1 - Q_d \leq 0 \mid level < 0$  then
19:    $level \leftarrow 0$ 
20: end if
21:  $R_e = list_{R_e}[level]$ 

```

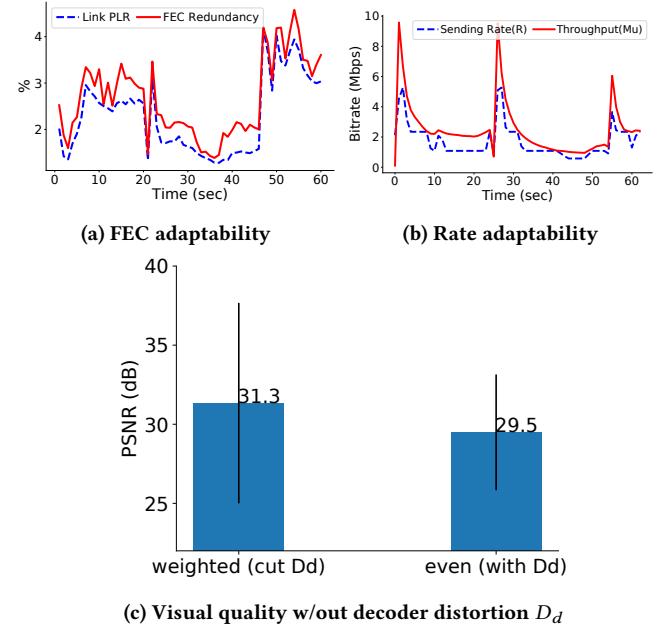


Figure 11: Illustration of redundancy and sending rate, and visual quality (PSNR) given a streaming of 1-minute gameplay video over emulated network (see Section 6.3). (a) FEC redundant information percentage (solid line) vs packet loss rate PLR (II) (dashed line) for frames over 40 packets, (b) sending rate vs network throughput, and (c) weighted protection from decoder distortion (cut D_d) vs equal protection (with D_d).

Given the heuristic algorithm, Figure 11 illustrates how adaptive are the redundancy and the total sending rate R as response to

Table 1: Baselines considered in the system evaluation

Protocol	Loss Recovery	Rate Control
TCP Cubic	NACK	Congestion Window
WebRTC	Hybrid NACK/FEC	GCC
BO [18, 19]	N/A	Buffer Occupancy
ESCOT [32]	GoP-level FEC	Throughput-latency based
Nebula	Frame-level FEC	

packet loss rate and throughput changes. By considering all the frames, however, the redundancy rate shown in Figure 11a increases reaching 9% on average, as described in Section C. Figure 11c illustrates the improvement in the visual quality (+1.8 dB) when applying weighted protection of the GoP frames (see Equation 5) to reduce the impact of error propagation and cut the decoder distortion D_d .

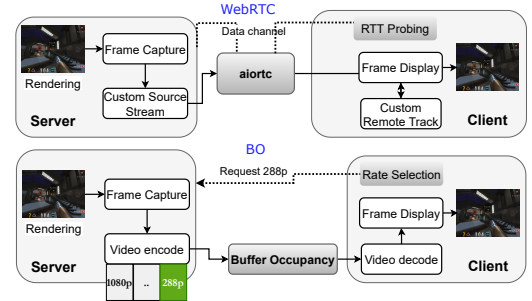
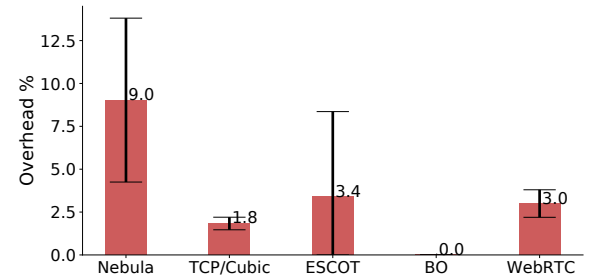
C BASELINES

Table 1 illustrated the baselines which are: the network transmission standard (TCP Cubic), the video streaming standard (WebRTC), Buffer Occupancy (BO)-based video streaming [18, 19], and GoP-based MCG (ESCOT) [32]. We integrate both BO and WebRTC to operate as streaming platform for mobile cloud gaming (see Figure 12). In both, we integrate the captured frames as source stream at the server, and a remote stream track at the client. We enable RTT probing in WebRTC using its data channel and we implement a queue-based playback buffer in BO based on which the Rate Selection component in the client requests the next rate from the server. Such integration of the baselines allows us not only to measure both visual quality and MTP on frame basis but also create reproducible experiments and findings.

D OVERHEAD

Overall, FEC-based approach tends to have a higher reliability overhead compared to ACK-based techniques that requires overhead equals to the packet loss rate. Figure 13 illustrates the reliability overhead of Nebula and the baselines's streaming over the emulated network described in Section 6.3.

Nebula's satisfactory visual quality is obtained at the expense of redundancy overhead. Nebula presents a redundancy rate of 9% on average, due to the per-frame encoding and the minimum redundancy. Each frame require a minimum of 1 redundant packet, and P-frames tend to be small (a couple of packets). Besides, unequally protecting the GoP frames leads to extra redundancy. As such, the overall overhead becomes relatively high, compared to GoP-level FEC coding. However, such overhead is compensated by the better throughput utilisation, minimizing its impact on the transmission.

**Figure 12: WebRTC's and BO's experimental frameworks.****Figure 13: Redundant packet as reliability overhead for each solution. Nebula has the highest, followed by GoP-level FEC (ESCOT), then hybrid FEC/NACK (WebRTC), and finally NACK only (TCP/CUBIC), while BO has none.**